

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 5

«Процедуры, функции, триггеры в PostgreSQL»

Обучающийся: Колпаков Артем Сергеевич

Факультет: Прикладной информатики

Группа: К3239

Преподаватель: Говорова Марина Михайловна

Санкт-Петербург

Содержание

1 Цель работы.....	2
2 Практическое задание.....	2
3 Процедуры.....	2
3.1 Процедура 1: Пассажиры, которые заказывали такси в заданном временном интервале. 2	
3.2 Процедура 2: Автомобили с пробегом больше 250000 и без ТО в текущем году.....	3
3.3 Процедура 3: Суммарный доход таксопарка за истекший месяц.....	4
4 Триггеры.....	4
4.1 Создаем таблицу для логов.....	4
4.2 Создаём триггерную функцию.....	4
4.3 Триггер 1 - логирование заказов.....	6
4.4 Триггер 2 - логирование пассажиров.....	7
4.5 Триггер 3 - логирование сотрудников.....	7
4.6 Триггер 4 - логирование машин.....	8
4.7 Триггер 5 - логирование тарифов.....	8
4.8 Триггер 6 - логирование оплаты.....	9
4.9 Триггер 7 - логирование времени работы сотрудника.....	9
5 Задание на доп баллы: Модификация триггера.....	11
5.1 Модификация триггерной функции.....	11
5.2 Создание триггера.....	12
5.3 Проверка триггера.....	12
6 Выводы.....	13

1 Цель работы

Овладеть практическими создания и использования процедур, функций и триггеров в базе данных PostgreSQL.

2 Практическое задание

1. Создать 3 процедуры для индивидуальной БД согласно варианту (часть 4 ЛР 2). Допустимо использование IN/INOUT параметров. Допустимо создать авторские процедуры.
2. Создать триггеры для индивидуальной БД согласно варианту:
Вариант 2.1. 7 триггеров по ИЗ.
Вариант 2.2. 5 оригинальных триггеров + модификация триггерной функции Части 2 практического занятия доп. баллы).

Дополнительные баллы - 3:

Модифицировать триггер (триггерную функцию) на проверку корректности входа и выхода сотрудника (см. Практическое задание 1 Лабораторного практикума (Приложение)) с максимальным учетом «узких» мест некорректных данных по входу и выходу).

3 Процедуры

Ниже приведены реализации процедур согласно варианту №14.

3.1 Процедура 1: Пассажиры, которые заказывали такси в заданном временном интервале

```
CREATE OR REPLACE PROCEDURE taxi.get_passengers_by_time_interval(
    start_time TIMESTAMP,
    end_time TIMESTAMP
)
LANGUAGE plpgsql
AS $psql$
BEGIN
    DROP TABLE IF EXISTS result_passengers_by_time_interval;

    CREATE TEMP TABLE result_passengers_by_time_interval AS
    SELECT
        passenger.passenger_id,
        passenger.full_name,
        passenger.phone,
        passenger.email,
        orders.order_id,
        orders.call_datetime,
        orders.pickup_address,
        orders.destination_address,
        orders.order_status
```

```

FROM taxi.passenger
JOIN taxi.orders ON orders.passenger_id = passenger.passenger_id
WHERE orders.call_datetime BETWEEN start_time AND end_time;
END;
$psql$;

```

```

taxi_service_db=# CALL taxi.get_passengers_by_time_interval(
    '2026-04-11 18:22:51.069248',
    '2026-04-12 18:51:45.45134'
);
CALL
taxi_service_db=# SELECT * FROM result_passengers_by_time_interval;
 passenger_id | full_name | phone | email | order_id | call_datetime | pickup_address | destination_address | order_status
-----
1 | Иван Петров | +79990000001 | ivan.petrov@mail.ru | 4 | 2026-04-12 18:51:45.45133 | ул. Ленина, 10 | Невский проспект, 25 | done
(1 строка)
taxi_service_db=#

```

3.2 Процедура 2: Куда был доставлен пассажир по номеру телефона

```

CREATE OR REPLACE PROCEDURE taxi.get_passenger_destinations(
    passenger_phone VARCHAR
)
LANGUAGE plpgsql
AS $psql$
BEGIN
    DROP TABLE IF EXISTS result_passenger_destinations;

    CREATE TEMP TABLE result_passenger_destinations AS
    SELECT
        passenger.passenger_id,
        passenger.full_name,
        passenger.phone,
        orders.order_id,
        orders.call_datetime,
        orders.pickup_address,
        orders.destination_address,
        orders.distance_km,
        orders.order_cost,
        orders.order_status
    FROM taxi.passenger
    JOIN taxi.orders ON orders.passenger_id = passenger.passenger_id
    WHERE passenger.phone = passenger_phone;
END;
$psql$;

```

```

taxi_service_db=# CALL taxi.get_passenger_destinations('+79990000001');
SELECT * FROM result_passenger_destinations;
NOTICE: table "result_passenger_destinations" does not exist, skipping
CALL
 passenger_id | full_name | phone | order_id | call_datetime | pickup_address | destination_address | distance_km | order_cost | order_status
-----
1 | Иван Петров | +79990000001 | 1 | 2026-04-11 18:22:51.069247 | Невский пр., 15 | Московский вокзал | 6.50 | 220.00 | done
1 | Иван Петров | +79990000001 | 4 | 2026-04-12 18:51:45.45133 | ул. Ленина, 10 | Невский проспект, 25 | 12.50 | 650.00 | done
(2 строки)

```

3.3 Процедура 3: Суммарный доход таксопарка за истекший месяц

```
CREATE OR REPLACE PROCEDURE taxi.get_last_month_income()
LANGUAGE plpgsql
AS $psql$
BEGIN
    DROP TABLE IF EXISTS result_last_month_income;

    CREATE TEMP TABLE result_last_month_income AS
    SELECT
        SUM(orders.order_cost) AS income
    FROM taxi.orders
    WHERE orders.call_datetime >= DATE_TRUNC('month', CURRENT_DATE -
INTERVAL '1 month')
        AND orders.call_datetime < DATE_TRUNC('month', CURRENT_DATE);
END;
$psql$;
```

```
taxi_service_db=# CALL taxi.get_last_month_income();

SELECT * FROM result_last_month_income;
NOTICE: table "result_last_month_income" does not exist, skipping
CALL
 income
-----
 1710.00
(1 строка)
```

4 Триггеры

4.1 Создаем таблицу для логов

```
CREATE TABLE taxi.logs (
    id SERIAL PRIMARY KEY,
    table_name VARCHAR(50),
    operation VARCHAR(20),
    row_id INT,
    text TEXT,
    added TIMESTAMP WITHOUT TIME ZONE DEFAULT NOW()
);
```

4.2 Создаём триггерную функцию

```
CREATE OR REPLACE FUNCTION taxi.add_to_log() RETURNS TRIGGER AS $$
DECLARE
    mstr VARCHAR(30);
```

```

astr VARCHAR(100);
retstr VARCHAR(254);
row_id INT;
BEGIN
  IF TG_OP = 'INSERT' THEN
    mstr := 'Add row in table ';
    astr := TG_TABLE_NAME;
    retstr := mstr || astr;

    IF TG_TABLE_NAME = 'orders' THEN
      row_id := NEW.order_id;
    ELSIF TG_TABLE_NAME = 'passenger' THEN
      row_id := NEW.passenger_id;
    ELSIF TG_TABLE_NAME = 'employee' THEN
      row_id := NEW.employee_id;
    ELSIF TG_TABLE_NAME = 'car' THEN
      row_id := NEW.car_id;
    ELSIF TG_TABLE_NAME = 'tariff' THEN
      row_id := NEW.tariff_id;
    ELSIF TG_TABLE_NAME = 'payment_card' THEN
      row_id := NEW.card_id;
    ELSIF TG_TABLE_NAME = 'employee_work_schedule' THEN
      row_id := NEW.work_schedule_id;
    END IF;

    INSERT INTO taxi.logs(table_name, operation, row_id, text,
added)
VALUES (TG_TABLE_NAME, TG_OP, row_id, retstr, NOW());

    RETURN NEW;

  ELSIF TG_OP = 'UPDATE' THEN
    mstr := 'Update row in table ';
    astr := TG_TABLE_NAME;
    retstr := mstr || astr;

    IF TG_TABLE_NAME = 'orders' THEN
      row_id := NEW.order_id;
    ELSIF TG_TABLE_NAME = 'passenger' THEN
      row_id := NEW.passenger_id;
    ELSIF TG_TABLE_NAME = 'employee' THEN
      row_id := NEW.employee_id;
    ELSIF TG_TABLE_NAME = 'car' THEN
      row_id := NEW.car_id;
    ELSIF TG_TABLE_NAME = 'tariff' THEN
      row_id := NEW.tariff_id;

```

```

        ELSIF TG_TABLE_NAME = 'payment_card' THEN
            row_id := NEW.card_id;
        ELSIF TG_TABLE_NAME = 'employee_work_schedule' THEN
            row_id := NEW.work_schedule_id;
        END IF;

        INSERT INTO taxi.logs(table_name, operation, row_id, text,
added)
        VALUES (TG_TABLE_NAME, TG_OP, row_id, retstr, NOW());

        RETURN NEW;

    ELSIF TG_OP = 'DELETE' THEN
        mstr := 'Remove row in table ';
        astr := TG_TABLE_NAME;
        retstr := mstr || astr;

        IF TG_TABLE_NAME = 'orders' THEN
            row_id := OLD.order_id;
        ELSIF TG_TABLE_NAME = 'passenger' THEN
            row_id := OLD.passenger_id;
        ELSIF TG_TABLE_NAME = 'employee' THEN
            row_id := OLD.employee_id;
        ELSIF TG_TABLE_NAME = 'car' THEN
            row_id := OLD.car_id;
        ELSIF TG_TABLE_NAME = 'tariff' THEN
            row_id := OLD.tariff_id;
        ELSIF TG_TABLE_NAME = 'payment_card' THEN
            row_id := OLD.card_id;
        ELSIF TG_TABLE_NAME = 'employee_work_schedule' THEN
            row_id := OLD.work_schedule_id;
        END IF;

        INSERT INTO taxi.logs(table_name, operation, row_id, text,
added)
        VALUES (TG_TABLE_NAME, TG_OP, row_id, retstr, NOW());

        RETURN OLD;
    END IF;
END;
$$ LANGUAGE plpgsql;

```

4.3 Триггер 1 - логирование заказов

```
CREATE TRIGGER t_orders_log
AFTER INSERT OR UPDATE OR DELETE ON taxi.orders
FOR EACH ROW EXECUTE PROCEDURE taxi.add_to_log();
```

```
taxi_service_db=# CREATE TRIGGER t_orders_log
AFTER INSERT OR UPDATE OR DELETE ON taxi.orders
FOR EACH ROW EXECUTE PROCEDURE taxi.add_to_log();
CREATE TRIGGER
taxi_service_db=# UPDATE taxi.orders
SET order_status = 'completed'
WHERE order_id = 1;
UPDATE 1
taxi_service_db=# SELECT *
FROM taxi.logs;
 id | table_name | operation | row_id |          text          |          added
-----+-----+-----+-----+-----+-----
  1 | orders    | UPDATE   |      1 | Update row in table orders | 2026-05-11 17:53:44.7961
(1 строка)
```

4.4 Триггер 2 - логирование пассажиров

```
CREATE TRIGGER t_passenger_log
AFTER INSERT OR UPDATE OR DELETE ON taxi.passenger
FOR EACH ROW EXECUTE PROCEDURE taxi.add_to_log();
```

```
taxi_service_db=# UPDATE taxi.passenger
SET notes = 'Проверка триггера пассажиров'
WHERE passenger_id = 1;

SELECT
    id,
    table_name,
    operation,
    row_id,
    text,
    added
FROM taxi.logs
ORDER BY id DESC
LIMIT 5;
UPDATE 1
 id | table_name | operation | row_id |          text          |          added
-----+-----+-----+-----+-----+-----
  2 | passenger  | UPDATE   |      1 | Update row in table passenger | 2026-05-11 17:57:42.17197
  1 | orders    | UPDATE   |      1 | Update row in table orders    | 2026-05-11 17:53:44.7961
(2 строки)
```

4.5 Триггер 3 - логирование сотрудников

```
CREATE TRIGGER t_employee_log
AFTER INSERT OR UPDATE OR DELETE ON taxi.employee
FOR EACH ROW EXECUTE PROCEDURE taxi.add_to_log();
```



```

taxi_service_db=# INSERT INTO taxi.employee (
    employee_category_id,
    employee_position_code,
    phone,
    address,
    full_name,
    passport_data,
    status
)
VALUES (
    1,
    1,
    '+79999999999',
    'Тестовый адрес',
    'Тестовый Сотрудник',
    '0000 000000',
    'active'
);

SELECT
    id,
    table_name,
    operation,
    row_id,
    text,
    added
FROM taxi.logs
ORDER BY id DESC
LIMIT 5;
INSERT 0 1

```

id	table_name	operation	row_id	text	added
3	employee	INSERT	4	Add row in table employee	2026-05-11 17:59:13.771014
2	passenger	UPDATE	1	Update row in table passenger	2026-05-11 17:57:42.17197
1	orders	UPDATE	1	Update row in table orders	2026-05-11 17:53:44.7961

(3 строки)

4.6 Триггер 4 - логирование машин

```

CREATE TRIGGER t_car_log
AFTER INSERT OR UPDATE OR DELETE ON taxi.car
FOR EACH ROW EXECUTE PROCEDURE taxi.add_to_log();

```

```

taxi_service_db=# UPDATE taxi.car
SET mileage = mileage + 1
WHERE car_id = 1;

SELECT
    id,
    table_name,
    operation,
    row_id,
    text,
    added
FROM taxi.logs
ORDER BY id DESC
LIMIT 5;
UPDATE 1

```

id	table_name	operation	row_id	text	added
4	car	UPDATE	1	Update row in table car	2026-05-11 18:00:12.638899
3	employee	INSERT	4	Add row in table employee	2026-05-11 17:59:13.771014
2	passenger	UPDATE	1	Update row in table passenger	2026-05-11 17:57:42.17197
1	orders	UPDATE	1	Update row in table orders	2026-05-11 17:53:44.7961

(4 строки)

4.7 Триггер 5 - логирование тарифов

```
CREATE TRIGGER t_tariff_log
AFTER INSERT OR UPDATE OR DELETE ON taxi.tariff
FOR EACH ROW EXECUTE PROCEDURE taxi.add_to_log();
```

```
taxi_service_db=# UPDATE taxi.tariff
SET waiting_fee_per_minute = waiting_fee_per_minute + 1
WHERE tariff_id = 1;
```

```
SELECT
    id,
    table_name,
    operation,
    row_id,
    text,
    added
FROM taxi.logs
ORDER BY id DESC
LIMIT 5;
UPDATE 1
```

id	table_name	operation	row_id	text	added
5	tariff	UPDATE	1	Update row in table tariff	2026-05-11 18:01:00.923743
4	car	UPDATE	1	Update row in table car	2026-05-11 18:00:12.638899
3	employee	INSERT	4	Add row in table employee	2026-05-11 17:59:13.771014
2	passenger	UPDATE	1	Update row in table passenger	2026-05-11 17:57:42.17197
1	orders	UPDATE	1	Update row in table orders	2026-05-11 17:53:44.7961

(5 строк)

4.8 Триггер 6 - логирование оплаты

```
CREATE TRIGGER t_payment_card_log
AFTER INSERT OR UPDATE OR DELETE ON taxi.payment_card
FOR EACH ROW EXECUTE PROCEDURE taxi.add_to_log();
```

```
taxi_service_db=# UPDATE taxi.payment_card
SET holder_name = 'IVAN PETROV TEST'
WHERE card_id = 1;
```

```
SELECT
    id,
    table_name,
    operation,
    row_id,
    text,
    added
FROM taxi.logs
ORDER BY id DESC
LIMIT 5;
UPDATE 1
```

id	table_name	operation	row_id	text	added
6	payment_card	UPDATE	1	Update row in table payment_card	2026-05-11 18:01:49.923741
5	tariff	UPDATE	1	Update row in table tariff	2026-05-11 18:01:00.923743
4	car	UPDATE	1	Update row in table car	2026-05-11 18:00:12.638899
3	employee	INSERT	4	Add row in table employee	2026-05-11 17:59:13.771014
2	passenger	UPDATE	1	Update row in table passenger	2026-05-11 17:57:42.17197

(5 строк)

4.9 Триггер 7 - логирование времени работы сотрудника

```
CREATE TRIGGER t_employee_work_schedule_log
AFTER INSERT OR UPDATE OR DELETE ON taxi.employee_work_schedule
```

```
FOR EACH ROW EXECUTE PROCEDURE taxi.add_to_log();
```

```
taxi_service_db=# UPDATE taxi.employee_work_schedule  
SET status = 'working'  
WHERE work_schedule_id = 1;
```

```
SELECT  
  id,  
  table_name,  
  operation,  
  row_id,  
  text,  
  added
```

```
FROM taxi.logs  
ORDER BY id DESC  
LIMIT 5;
```

```
UPDATE 1
```

id	table_name	operation	row_id	text	added
7	employee_work_schedule	UPDATE	1	Update row in table employee_work_schedule	2026-05-11 18:03:05.281603
6	payment_card	UPDATE	1	Update row in table payment_card	2026-05-11 18:01:49.923741
5	tariff	UPDATE	1	Update row in table tariff	2026-05-11 18:01:00.923743
4	car	UPDATE	1	Update row in table car	2026-05-11 18:00:12.638899
3	employee	INSERT	4	Add row in table employee	2026-05-11 17:59:13.771014

```
(5 строк)
```

5 Задание на доп баллы: Модификация триггера

5.1 Модификация триггерной функции

Исходный триггер из методички просто проверяет, чтобы не было двух одинаковых событий подряд: двух входов или двух выходов.

Я расширил проверку и учел больше узких мест:

- первое событие сотрудника не может быть выходом
- нельзя делать два входа подряд
- нельзя делать два выхода подряд
- нельзя вставлять событие раньше последнего события сотрудника
- нельзя вставлять событие в будущем

```
CREATE OR REPLACE FUNCTION emp_time.fn_check_time_punch() RETURNS
TRIGGER AS $psql$
DECLARE
    last_is_out_punch BOOLEAN;
    last_punch_time TIMESTAMP;
BEGIN
    SELECT
        time_punch.is_out_punch,
        time_punch.punch_time
    INTO
        last_is_out_punch,
        last_punch_time
    FROM emp_time.time_punch
    WHERE time_punch.employee_id = NEW.employee_id
    ORDER BY time_punch.punch_time DESC
    LIMIT 1;

    IF NEW.punch_time > NOW() THEN
        RAISE NOTICE 'Ошибка: время входа или выхода не может быть в
будущем';
        RETURN NULL;
    END IF;

    IF last_punch_time IS NULL AND NEW.is_out_punch = TRUE THEN
        RAISE NOTICE 'Ошибка: первым событием сотрудника должен быть
вход, а не выход';
        RETURN NULL;
    END IF;

    IF last_punch_time IS NOT NULL AND NEW.punch_time <=
last_punch_time THEN
        RAISE NOTICE 'Ошибка: новое время должно быть больше
```

```

последнего времени входа или выхода';
    RETURN NULL;
END IF;

    IF last_is_out_punch = NEW.is_out_punch THEN
        RAISE NOTICE 'Ошибка: нельзя добавить два входа или два выхода
поряд';
        RETURN NULL;
    END IF;

    RETURN NEW;
END;
$psql$ LANGUAGE plpgsql;

```

5.2 Создание триггера

```

CREATE TRIGGER check_time_punch
BEFORE INSERT ON emp_time.time_punch
FOR EACH ROW EXECUTE PROCEDURE emp_time.fn_check_time_punch();

```

5.3 Проверка триггера

1. Корректный вход

```

taxi_service_db=# INSERT INTO emp_time.time_punch(employee_id, is_out_punch, punch_time)
VALUES (1, FALSE, '2021-01-01 10:00:00');
INSERT 0 1
taxi_service_db=# SELECT *
FROM emp_time.time_punch;
 id | employee_id | is_out_punch |      punch_time
-----+-----+-----+-----
  1 |          1 | f           | 2021-01-01 10:00:00
(1 строка)

```

2. Некорректный повторный вход

```

taxi_service_db=# INSERT INTO emp_time.time_punch(employee_id, is_out_punch, punch_time)
VALUES (1, FALSE, '2021-01-01 10:30:00');
NOTICE:  Ошибка: нельзя добавить два входа или два выхода подряд
INSERT 0 0
taxi_service_db=#

```

3. Корректный выход

```

taxi_service_db=# INSERT INTO emp_time.time_punch(employee_id, is_out_punch, punch_time)
VALUES (1, FALSE, '2021-01-01 10:30:00');
NOTICE: Ошибка: нельзя добавить два входа или два выхода подряд
INSERT 0 0
taxi_service_db=# INSERT INTO emp_time.time_punch(employee_id, is_out_punch, punch_time)
VALUES (1, TRUE, '2021-01-01 11:30:00');
INSERT 0 1
taxi_service_db=# SELECT *
FROM emp_time.time_punch;
 id | employee_id | is_out_punch |      punch_time
-----+-----+-----+-----
  1 |           1 | f           | 2021-01-01 10:00:00
  3 |           1 | t           | 2021-01-01 11:30:00
(2 строки)

```

4. Некорректный повторный выход

```

taxi_service_db=# INSERT INTO emp_time.time_punch(employee_id, is_out_punch, punch_time)
VALUES (1, TRUE, '2021-01-01 12:00:00');
NOTICE: Ошибка: нельзя добавить два входа или два выхода подряд
INSERT 0 0
taxi_service_db=#

```

5. Некорректное время раньше последнего события

```

taxi_service_db=# INSERT INTO emp_time.time_punch(employee_id, is_out_punch, punch_time)
VALUES (1, FALSE, '2021-01-01 11:00:00');
NOTICE: Ошибка: новое время должно быть больше последнего времени входа или выхода
INSERT 0 0
taxi_service_db=#

```

6. Корректный следующий вход

```

taxi_service_db=# INSERT INTO emp_time.time_punch(employee_id, is_out_punch, punch_time)
VALUES (1, FALSE, '2021-01-01 12:30:00');
INSERT 0 1
taxi_service_db=#

```

7. Некорректный первый выход для нового сотрудника

```

taxi_service_db=# INSERT INTO emp_time.employee(username)
VALUES ('Иван');
INSERT 0 1
taxi_service_db=# INSERT INTO emp_time.time_punch(employee_id, is_out_punch, punch_time)
VALUES (2, TRUE, '2021-01-01 09:00:00');
NOTICE: Ошибка: первым событием сотрудника должен быть вход, а не выход
INSERT 0 0
taxi_service_db=#

```

6 Выводы

В ходе выполнения лабораторной работы были получены практические навыки создания и использования процедур, функций и триггеров в PostgreSQL. Для индивидуальной базы данных “Служба заказа такси” были разработаны три хранимые процедуры, позволяющие получать сведения о пассажирах за заданный временной интервал, выводить маршруты пассажира по номеру телефона и рассчитывать суммарный доход таксопарка за истекший месяц.

Также была реализована система логирования действий в базе данных. Для этого была создана таблица логов, триггерная функция и семь триггеров для основных таблиц базы данных. Проверка показала, что при выполнении операций INSERT, UPDATE и DELETE информация об изменениях успешно записывается в таблицу логов.

Дополнительно была модифицирована триггерная функция проверки корректности входа и выхода сотрудника. В результате были обработаны возможные случаи некорректных данных: повторный вход или выход, выход без предварительного входа,

некорректное время события и попытка указать время в будущем.

Таким образом, в работе были закреплены навыки создания процедур, триггерных функций и триггеров, а также проверки их работы через консоль `psql`.